Week 7 Worked Companion

Using the worked companion

Attempt the student exercises independently first. This companion contains hints and complete solutions. Read only as far as needed, then return to your attempt.

Examples are original teaching examples. Follow each official link for the source problem and its complete contract.

Student exercises

Instructor guide

Download Week 7 code

198 · Staged hints

LeetCode 198: House Robber · Medium

H1. Which prefix optimum may be combined with selecting the current position?

H2. Which old states are needed before overwriting?

H3. D[i] is the best prefix of length i. D[0]=0, D[1]=nums[0]; for i>=2, D[i]=max(D[i-1],D[i-2]+nums[i-1]). The code starts both rolling states at zero and applies the update from the first value.

198 · Correctness and cost

Every optimum either selects the current item or skips it. Each case leaves an optimal prefix, so the recurrence covers all optima.

n is length. O(n) time, O(1) auxiliary space. Reconstructing selected indices needs additional state.

198 · Counterexamples and checks

Summing only even or only odd positions misses optimum 4 for [2,1,1,2].

```
nums=[3,1,4,2] → 7
```

Predict the state and return value before running this example. Then check the boundary cases above.

198 · Implementation

```
def rob(nums):
    previous_two = previous_one = 0
    for value in nums:
        previous_two, previous_one = previous_one, max(previous_one, previous_two + value)
    return previous_one
```

322 · Staged hints

LeetCode 322: Coin Change · Medium

H1. Does dp[a] count ways or coins?

H2. What false candidate appears if an impossible amount starts at zero?

H3. Set dp[0]=0 and other states to infinity; fill increasing amounts using min(dp[a-c]+1).

322 · Correctness and cost

Removing the last coin c from an optimum leaves an optimum for a-c. Comparing all final coins finds the minimum; positive values ensure dependencies were filled earlier.

A is amount,m coin types. O(Am) time,O(A) space; pseudopolynomial in the bit length of A.

322 · Counterexamples and checks

Largest-first chooses 4+1+1 instead of 3+3 for [1,3,4],6. [2],3 gives -1; amount zero gives zero.

```
coins=[1,3,4], amount=6 → 2
```

Predict the state and return value before running this example. Then check the boundary cases above.

322 · Implementation

```
def coin_change(coins, amount):
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0
    for total in range(1, amount + 1):
        for coin in coins:
            if coin <= total:
                dp[total] = min(dp[total], dp[total - coin] + 1)
    return -1 if dp[amount] == float("inf") else dp[amount]
```

62 · Staged hints

LeetCode 62: Unique Paths · Medium

H1. Which cells can immediately precede a destination?

H2. Are updates independent when rows share one list object?

H3. Initialize first row/column to 1, add top and left counts, and allocate each row independently.

62 · Correctness and cost

Every path arrives from top or left. These disjoint classes cover all paths, so counts add; row order respects dependencies.

R,C are dimensions. O(RC) time/space under the bounded-answer model; arbitrary-size integer bit costs are separate.

62 · Counterexamples and checks

`[[1]*C]*R` aliases rows; use a comprehension creating each row.

```
rows=2, cols=4 → 4
```

Predict the state and return value before running this example. Then check the boundary cases above.

62 · Implementation

```
def unique_paths(rows, cols):
    dp = [[1] * cols for _ in range(rows)]
    for r in range(1, rows):
        for c in range(1, cols):
            dp[r][c] = dp[r-1][c] + dp[r][c-1]
    return dp[-1][-1]
```

1143 · Staged hints

LeetCode 1143: Longest Common Subsequence · Medium

H1. What subproblem do prefix lengths i,j define?

H2. If final characters differ, which end can be skipped?

H3. Equal final characters give diagonal+1; otherwise take max(top,left); empty prefixes give zero.

1143 · Correctness and cost

Equal final characters can be paired in an optimum. Unequal ends cannot both be the final match, so at least one is skipped; maximize over both cases.

n,m are string lengths. O(nm) time/space; this returns length, not a reconstructed sequence.

1143 · Counterexamples and checks

A subsequence preserves character order while allowing characters to be skipped. Skipping b in “abc” gives “ac”, so their longest common subsequence has length 2.

```
first="abca", second="aca" → 3
```

Predict the state and return value before running this example. Then check the boundary cases above.

1143 · Implementation

```
def lcs(first, second):
    rows, cols = len(first), len(second)
    dp = [[0] * (cols + 1) for _ in range(rows + 1)]
    for r in range(1, rows + 1):
        for c in range(1, cols + 1):
            if first[r-1] == second[c-1]:
                dp[r][c] = dp[r-1][c-1] + 1
            else:
                dp[r][c] = max(dp[r-1][c], dp[r][c-1])
    return dp[rows][cols]
```

Execution and sources

```
cd code
python demo_13.py
python demo_14.py
python verify.py
```

Standard-library imports are in `solutions.py`. On LeetCode, adapt the function to the official method signature.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

Checkpoint answers and reasoning

1. 4, selecting the first and last values. Comparing only odd and even positions misses this choice.
2. 2, using 3+3 rather than the greedy 4+1+1.
3. 1 and 4. The second row contains 1,2,3,4.
4. 2. A subsequence preserves order and may skip intermediate characters.

Concept-check explanations

The 1 in `dp[a-c]+1` counts the final added coin. Amount zero needs zero coins; unreachable amounts remain infinite.

Path counting adds the sizes of two disjoint sets distinguished by their final move. LCS maximizes possible lengths, so it cannot use the same addition rule.